# Recomendaciones y actividades sugeridas para el exito del laboratorio

**Redux avanzado**

Instalar reselect

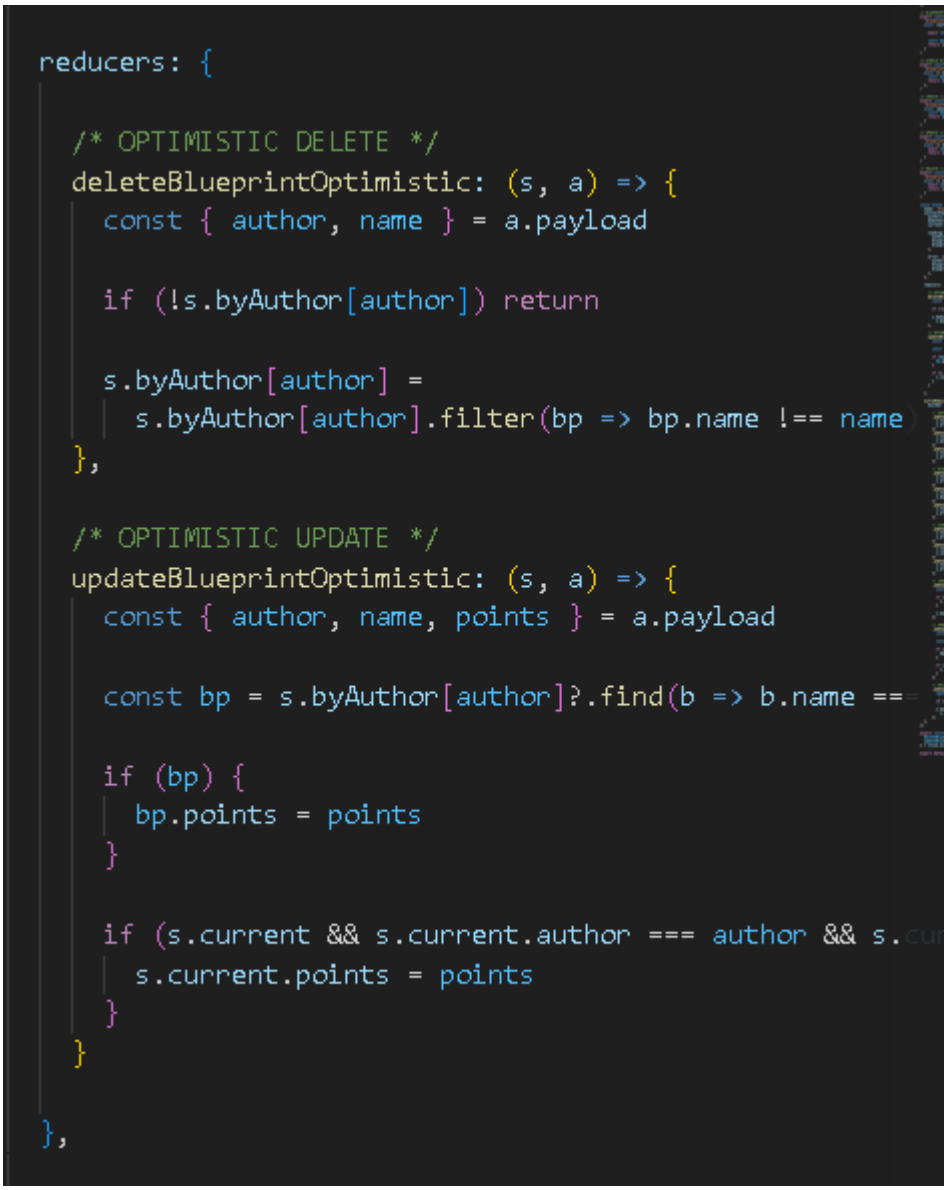
Redux Toolkit ya lo incluye, pero si no:

npm install reselect

mejora en el slice

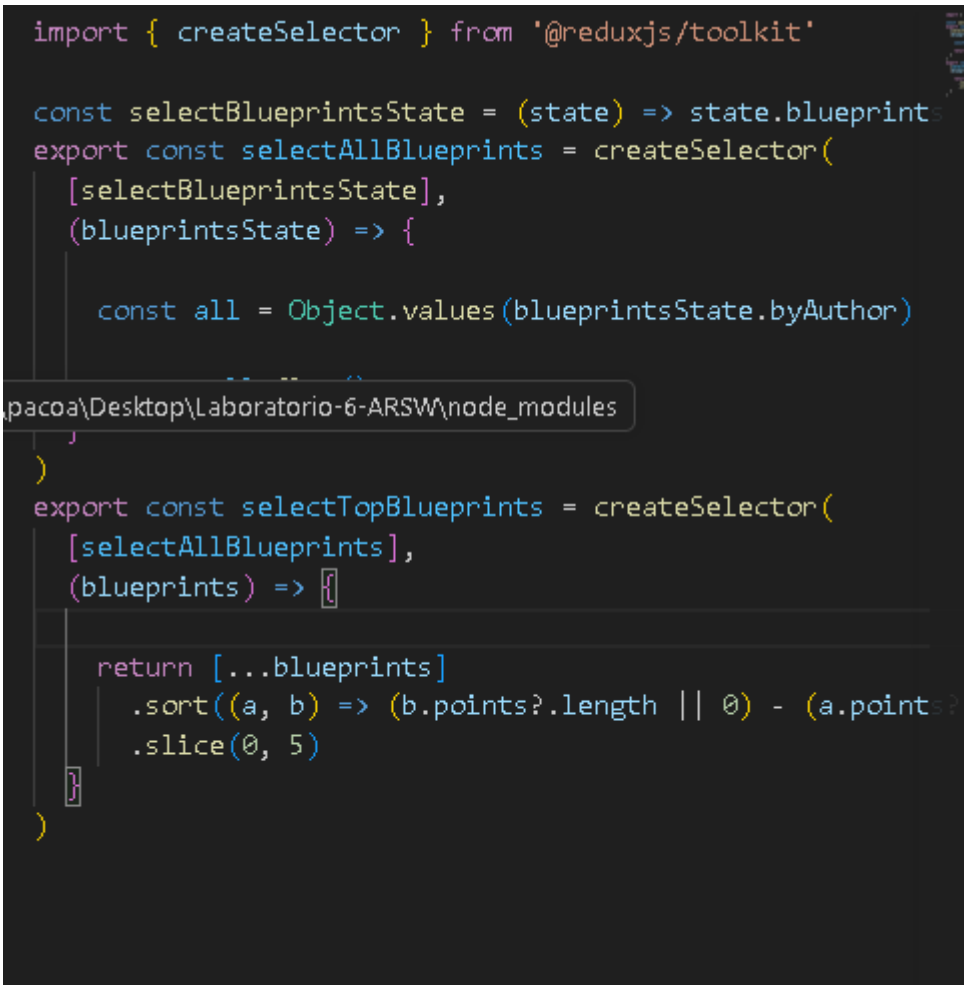
```
const slice = createSlice({
  name: 'blueprints',
  initialState: {
    authors: [],
    byAuthor: {},
    current: null,

    loading: {
      authors: false,
      blueprints: false,
      blueprint: false,
    },

    error: {
      authors: null,
      blueprints: null,
      blueprint: null,
    },
  },

  reducers: {
```

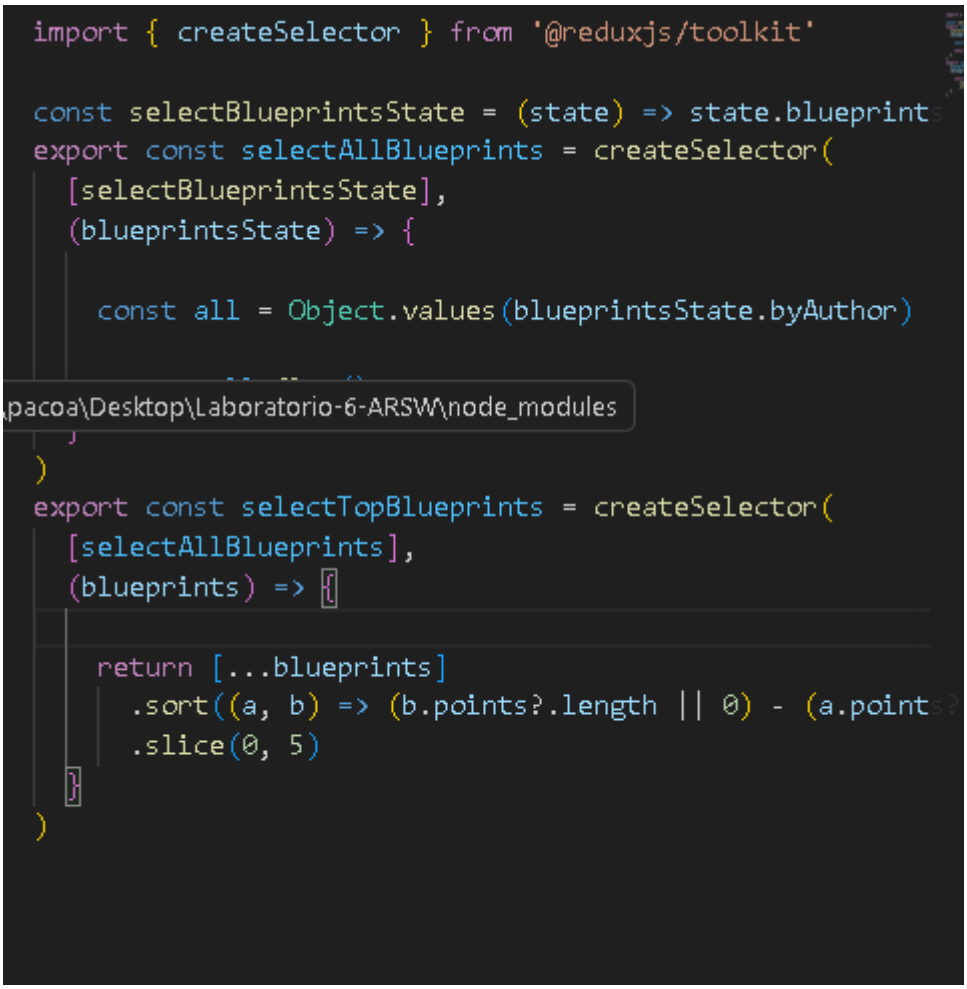
Y reducer



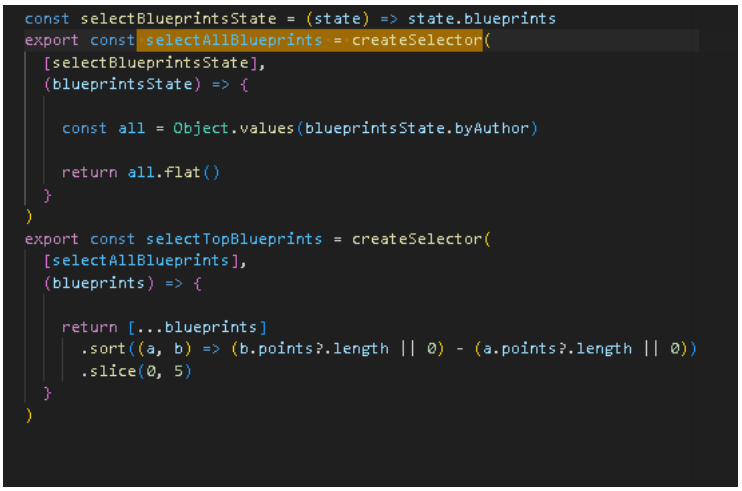
```
reducers: {

  /* OPTIMISTIC DELETE */
  deleteBlueprintOptimistic: (s, a) => {
    const { author, name } = a.payload

    if (!s.byAuthor[author]) return

    s.byAuthor[author] =
      s.byAuthor[author].filter(bp => bp.name !== name)
  },

  /* OPTIMISTIC UPDATE */
  updateBlueprintOptimistic: (s, a) => {
    const { author, name, points } = a.payload

    const bp = s.byAuthor[author]?.find(b => b.name ===

    if (bp) {
      bp.points = points
    }

    if (s.current && s.current.author === author && s.cu
      s.current.points = points
    }
  }

},
```

Luego se creo un memo selector


import { createSelector } from '@reduxjs/toolkit'
const selectBlueprintsState = (state) => state.blueprint
export const selectAllBlueprints = createSelector(
[selectBlueprintsState],
(blueprintsState) => {
const all = Object.values(blueprintsState.byAuthor)
pacoa\Desktop\Laboratorio-6-ARSW\node_modules
)
export const selectTopBlueprints = createSelector(
[selectAllBlueprints],
(blueprints) => {
return [...blueprints]
.sort((a, b) => (b.points?.length || 0) - (a.point
.slice(0, 5)
}
)




Obtener todos los blueprints y Top 5 por cantidad de puntos




En la UI se implemento un sistema de mensajes de si esta cargando el get o un mensaje de error si no lo encuentra, también se implemento el top 5 de blueprints que tengan más puntos

Error loading blueprints: Request failed with status code 404



**Rutas protegidas**

**Si hay token → deja entrar**
**Si no hay token → redirige al login**

```jsx
import { Navigate } from 'react-router-dom'

export default function PrivateRoute({ children }) {

  const token = localStorage.getItem('token')

  if (!token || token === "null" || token === "undefined") {
    return <Navigate to="/login" replace />
  }

  return children
}
```

```jsx
App.jsx > App
import { NavLink, Route, Routes } from 'react-router-dom'
import BlueprintsPage from './pages/BlueprintsPage.jsx'
import BlueprintDetailPage from './pages/BlueprintDetailPage.jsx'
import LoginPage from './pages/LoginPage.jsx'
import NotFound from './pages/NotFound.jsx'
import PrivateRoute from './routes/PrivateRoute.jsx'

export default function App() {
  return (
    <div className="container">

      <header>
        <h1>ECI - Laboratorio de Blueprints en React</h1>

        <nav>
          <NavLink to="/" end>
            Blueprints
          </NavLink>

          <NavLink to="/login">
            Login
          </NavLink>
        </nav>

      </header>

      <Routes>

        <Route
          path="/"
          element={
            <PrivateRoute>
              <BlueprintsPage />
            </PrivateRoute>
          }
        />

        <Route
          path="/blueprints/:authory:name"
          element={
            <PrivateRoute>
              <BlueprintDetailPage />
            </PrivateRoute>
          }
        />

        <Route path="/login" element={<LoginPage />} />

        <Route path="*" element={<NotFound />} />

      </Routes>

    </div>
  )
}
```

Estos códigos hace que si al ejecutar el programa entra directamente a /blueprints/login debido a que no permite que un usuario no entre sin haber iniciado sesión al /blueprints/ si se intenta ir a esa ruta cambia automáticamente a login

**CRUD completo**

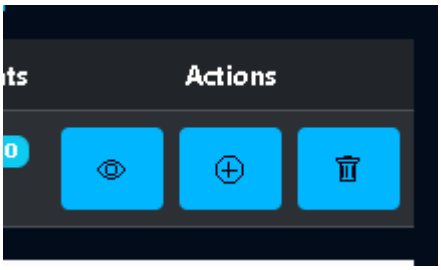
en el back se implemento los siguientes endpoints ya que no existían y tras ellos toda su lógica

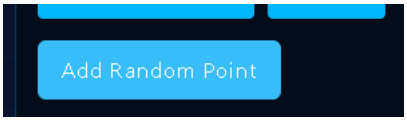
```java
                @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "404", description = "Blueprint no encontrad
    })
    @PutMapping("/{author}/{name}")
    public ResponseEntity<ApiResponse<Void>> updateBlueprint(
                @PathVariable String author,
                @PathVariable String name,
                @Valid @RequestBody UpdateBlueprintRequest req) {

        try {

                Blueprint bp = new Blueprint(author, name, req.points());
                services.updateBlueprint(bp);

                return ResponseEntity.ok(
                        new ApiResponse<>(200, "Blueprint updated", null));

        } catch (BlueprintNotFoundException e) {

                return ResponseEntity.status(HttpStatus.NOT_FOUND)
                        .body(new ApiResponse<>(404, e.getMessage(), null));
        }
    }

    // DELETE /blueprints/{author}/{name}
    @Operation(summary = "Eliminar un blueprint", description = "Elimina un blueprint existente identificado por autor y nombre")
    @ApiResponses({
                @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "200", description = "Blueprint eliminado co
                @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "404", description = "Blueprint no encontrad
    })
    @DeleteMapping("/{author}/{name}")
    public ResponseEntity<ApiResponse<Void>> delete(
                @PathVariable String author,
                @PathVariable String name) {

        try {

                services.deleteBlueprint(author, name);

                return ResponseEntity.ok(
                        new ApiResponse<>(200, "Blueprint deleted", null));

        } catch (BlueprintNotFoundException e) {

                return ResponseEntity.status(HttpStatus.NOT_FOUND)
                        .body(new ApiResponse<>(404, e.getMessage(), null));
        }
    }
```

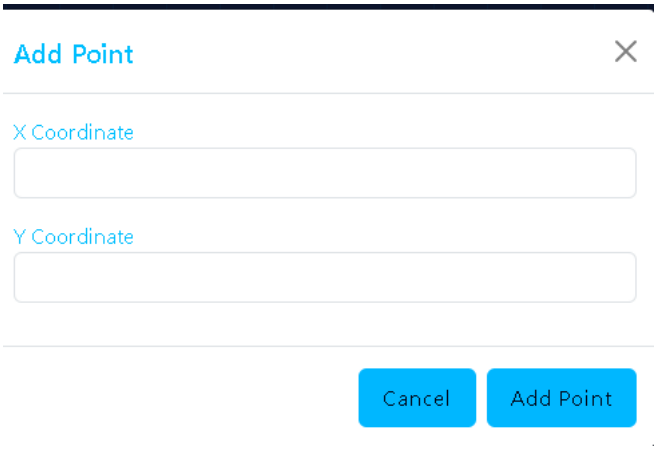
y luego se hizo que slice pudiera utilizar esos endpoints

```javascript
/* UPDATE */
export const updateBlueprint = createAsyncThunk(
  'blueprints/updateBlueprint',
  async ({ author, name, points }) => {
    await service.update(author, name, { points })
    return { author, name, points }
  }
)

/* DELETE */
export const deleteBlueprint = createAsyncThunk(
  'blueprints/deleteBlueprint',
  async ({ author, name }) => {
    await service.remove(author, name)
    return { author, name }
  }
)
```


Después en la UI se implementó tres botones uno que añade un punto en una ubicación aleatoria, otro de un punto en posición fija y por ultimo uno que elimina el plano




**Operación Endpoint Redux**

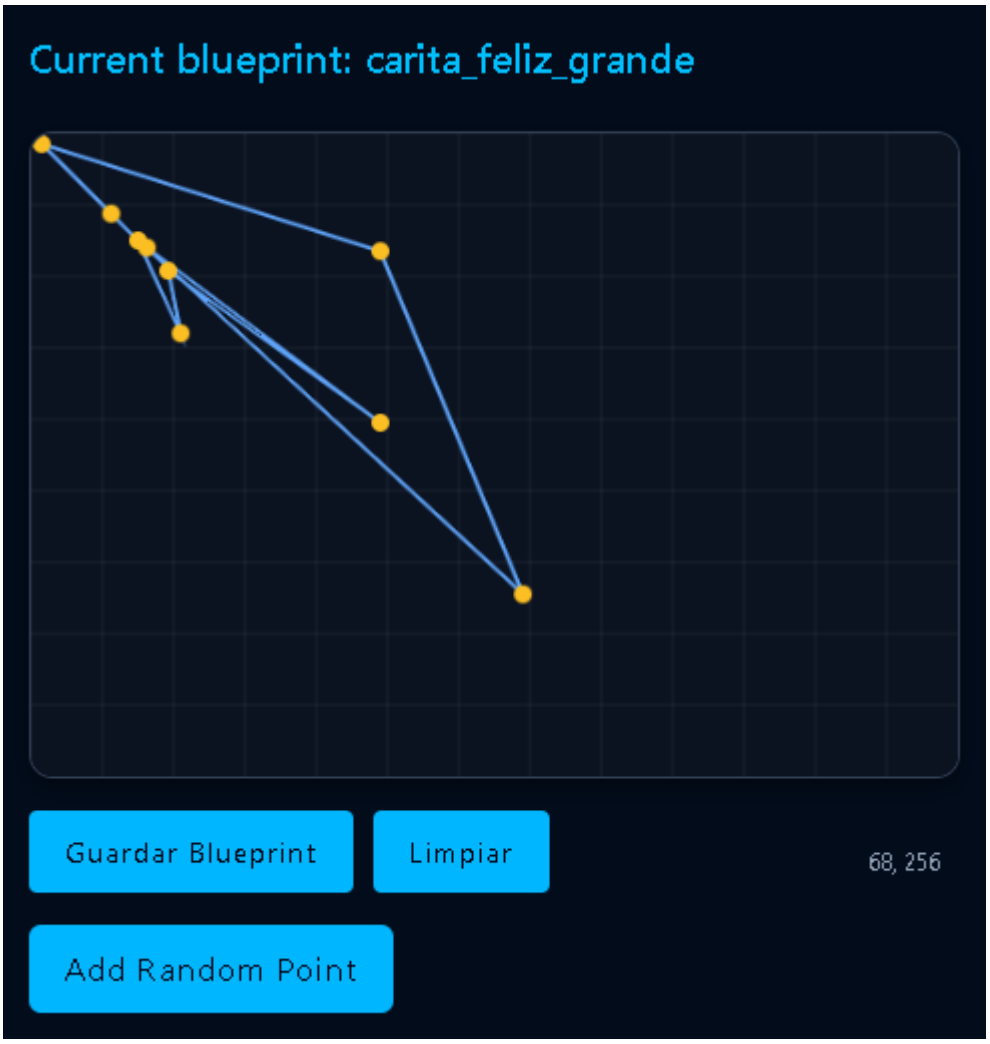
| | | |
|---|---|---|
| Create | POST | createBlueprint |
| Read | GET | fetchBlueprints |
| Update | PUT | updateBlueprint |
| Delete | DELETE | deleteBlueprint |

**Dibujo interactivo**

Permitir **agregar puntos haciendo click**
Mantener los puntos **localmente mientras se dibuja**
Tener un botón **Guardar** que envíe el blueprint al backend

```
// agregar punto con click
const handleClick = (e) => {

  const canvas = ref.current
  const rect = canvas.getBoundingClientRect()

  const scaleX = canvas.width / rect.width
  const scaleY = canvas.height / rect.height

  const x = Math.round((e.clientX - rect.left) * scaleX)
  const y = Math.round((e.clientY - rect.top) * scaleY)

  setLocalPoints((prev) => [...prev, { x, y }])
}

const handleMove = (e) => {

  const canvas = ref.current
  const rect = canvas.getBoundingClientRect()

  const scaleX = canvas.width / rect.width
  const scaleY = canvas.height / rect.height

  const x = Math.round((e.clientX - rect.left) * scaleX)
  const y = Math.round((e.clientY - rect.top) * scaleY)

  setMousePos({ x, y })
}
```

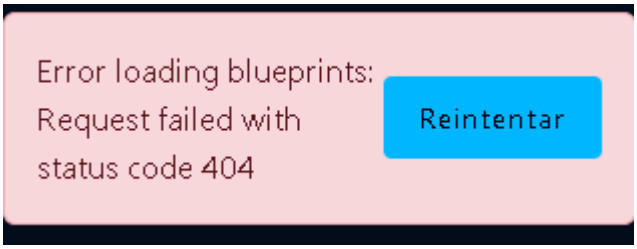
```
      <button
        className="btn btn-success btn-sm"
        onClick={handleSave}
      >
        Guardar Blueprint
      </button>

      <button
        className="btn btn-secondary btn-sm"
        onClick={handleClear}
      >
        Limpiar
      </button>

    </div>

  </div>
)
}
```




Se edito el canvas para que al hacer click en el plano se cree un punto y cuando le de a sabe lo guarda en el endpoint

**Errores y *Retr***

*En este punto se agregó un botón de reintentar cuando no falla una consulta, el mensaje de error se programo posteriormente antes de esté punto*



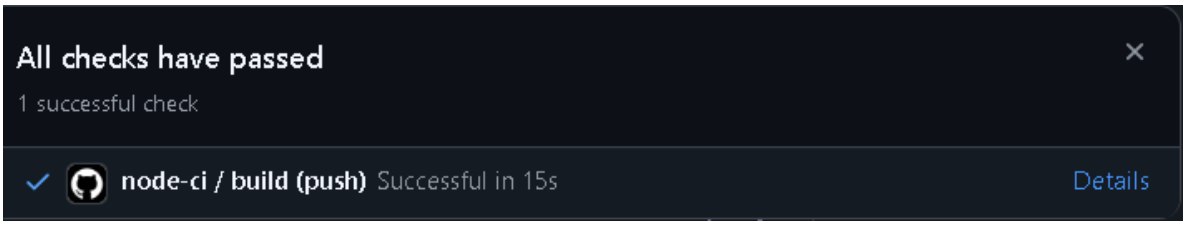
```
      <button
        className="btn btn-sm btn-light"
        onClick={retryFetch}
        disabled={loading.blueprints}
      >
        Reintentar
      </button>

    </div>
```

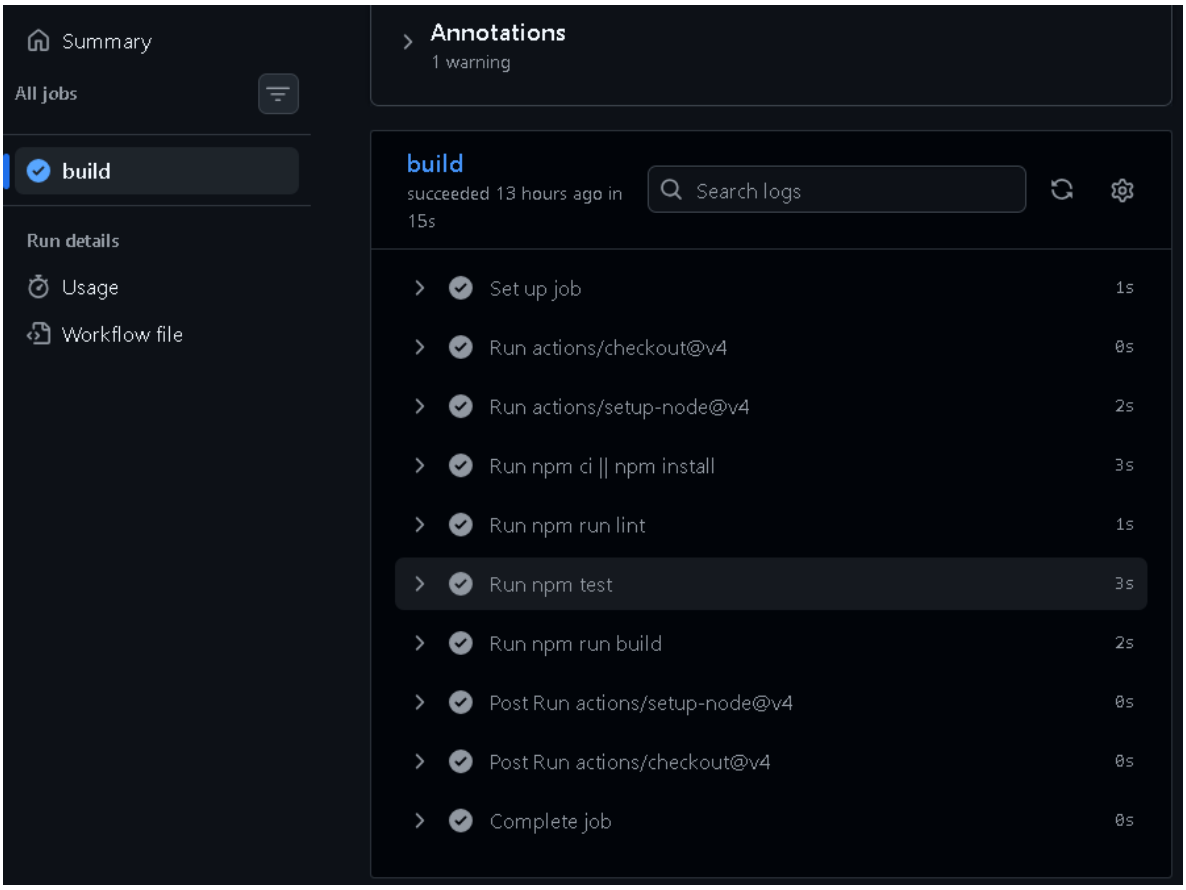
**Testing**

```
 ✓ tests/BlueprintsPage.test.jsx (1 test) 235ms

 Test Files  4 passed (4)
      Tests  12 passed (12)
   Start at  22:06:42
   Duration  6.85s (transform 903ms, setup 1.98s, collect 4.83s, tests 566ms, environment 14.35s, prepare 1.72s)
```

se modificaron los test para que cubrieran los casos modificados ya que los test que estaban previamente en requerimientos están desactualizados.

**CI/Lint/Format**






Al pasar las pruebas funciona correctamente el git actions que estaba programado en este repositorio

**Docker (opcional)**

se modifico todos los archivos de docker

```
# Build stage
FROM node:20-alpine AS build
WORKDIR /app

COPY package*.json ./
RUN npm ci || npm install

COPY . .
RUN npm run build

# Server stage
FROM node:20-alpine
WORKDIR /app

RUN npm i -g serve

COPY --from=build /app/dist ./dist

EXPOSE 4173

CMD ["serve", "-s", "dist", "-l", "4173"]
```

```
▷ Run Service
backend:
  build: ../Laboratorio-5-ARSW
  container_name: blueprints-backend
  ports:
    - "8080:8080"

▷ Run Service
web:
  build: .
  container_name: blueprints-frontend
  ports:
    - "5173:4173"
  environment:
    - VITE_API_BASE_URL=http://backend:8080/api
  depends_on:
    - backend
```

```dockerfile
# syntax=docker/dockerfile:1
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app

COPY pom.xml .
RUN mvn -B -f pom.xml dependency:go-offline

COPY src ./src
RUN mvn -B -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app

COPY --from=build /app/target/*.jar ./app.jar

EXPOSE 8080

ENTRYPOINT ["java","-jar","/app/app.jar"]
```

Y se levanto la imagen con el siguiente comando

docker compose up --build

y esto hace que funcionen los siguientes links

http://localhost:8080/actuator/health

http://localhost:8080/swagger-ui/index.html

http://localhost:5173